Module 4 – Introduction to DBMS

Introduction to SQL

Theory Questions:

1. What is SQL, and why is it essential in database management?

SQL, which stands for Structured Query Language is a standardized programming language specifically designed for managing and manipulating data in relational database management systems (RDBMS). It's the primary language used to interact with database that organize data into tables with rows and columns.

Why is SQL essential in database management?

SQL is crucial in database management for several key reasons:

1. Universality and Standardization:
    - SQL is the standard language for relational databases. This means that once you learn SQL, you can apply your knowledge across various RDBMS platforms like MySQL, PostgreSQL, Oracle Database, SQL Server, and more, with only minor syntax variations (dialects).
    - This standardization ensures consistency, portability of code, and a vast community for support and resources.
2. Efficient Data Management and Retrieval:
    - SQL provides powerful commands for interacting with data. You can quickly filter, sort, aggregate, and join data from multiple tables to extract meaningful insights.
    - It's designed to handle large volumes of data efficiently, allowing for near real-time data retrieval and manipulation, which is critical for business operations and analytics.
3. Data Integrity and Consistency:
    - SQL allows you to enforce data integrity rules and constraints (like primary keys, foreign keys, unique constraints). These rules ensure that data is accurate, consistent, and reliable, preventing errors and redundancies.
    - It helps maintain consistency across different users and applications accessing the same database.
4. Data Security:

- SQL provides mechanisms for managing user access and permissions. You can grant or revoke specific privileges to different users, ensuring that sensitive data is protected from unauthorized access.

5. Scalability:
    - SQL databases are designed to scale, handling increasing amounts of data and user traffic without significant performance degradation. This is vital for businesses of all sizes, from small applications to large enterprise systems.
6. Integration and Versatility:
    - SQL can be easily integrated with various programming languages (like Python, Java, C#) and tools, making it a versatile solution for data management and analysis in diverse applications.
    - It forms the backbone for many data-driven applications, from web applications to business intelligence systems.

2. Explain the difference between DBMS and RDBMS.

Features	DBMS (Database Management System)	RDBMS (Relational Database Management System)
Data Model	Can use various models (Hierarchical, network, file, etc.)	Specifically uses the **relational model** (tables, rows, columns)
Data Relationships	May not enforce explicit relationships between data	Enforces relationships between tables using **primary and foreign keys**
Data Integrity	Limited support for integrity constraints	Strong support for data integrity through constraints and **ACID properties**
Data Redundancy	Can have significant data redundancy	Aims to minimize data redundancy through **normalization**
User Access	Primarily supports single-user access (historically)	Designed for **multi-user concurrent access**
Query Language	May use proprietary query languages or no formal language	Primarily uses **SQL (Structured Query Language)**
Examples	File systems, XML databases, older database systems, MongoDB	MySQL, PostgreSQL, Oracle, SQL Server, IBM Db2

3. Describe the role of SQL in managing relational databases.

Here's a breakdown of SQL's key roles in managing relational databases:

1. **Data Definition (DDL - Data Definition Language):**
    - Creating Database Structures: SQL is used to define the schema (structure) of your database. This includes creating databases themselves, and then within those databases, defining tables, columns, data types, and constraints.
        - CREATE DATABASE: To create a new database.
        - CREATE TABLE: To define a new table with specific columns and their data types (e.g., VARCHAR, INT, DATE).
        - ALTER TABLE: To modify existing table structures (e.g., add/remove columns, change data types).
        - DROP TABLE: To delete entire tables.
2. Data Manipulation (DML - Data Manipulation Language):
    - Inserting Data: Adding new records (rows) into tables.
        - INSERT INTO: To add new data.
    - Retrieving Data (Querying): The most common and powerful use of SQL is to retrieve specific data from one or more tables. This involves filtering, sorting, joining, and aggregating data.
        - SELECT: The core command for querying data. It's often combined with:
            - WHERE: To filter rows based on conditions.
            - ORDER BY: To sort results.
            - GROUP BY: To aggregate data.
            - JOIN: To combine data from multiple related tables (e.g., INNER JOIN, LEFT JOIN, RIGHT JOIN).
    - Updating Data: Modifying existing records in tables.
        - UPDATE: To change data in existing rows.
    - Deleting Data: Removing records from tables.
        - DELETE FROM: To remove specific rows.
3. **Data Control (DCL - Data Control Language):**
    - Managing User Permissions: SQL provides commands to manage security and access control, specifying who can do what with the data.

        - GRANT: To give users specific permissions (e.g., SELECT, INSERT, UPDATE on certain tables).
        - REVOKE: To remove permissions from users.

4. **Transaction Control (TCL - Transaction Control Language):**
    - Ensuring Data Consistency (ACID Properties): SQL is vital for managing transactions, which are sequences of operations performed as a single logical unit of work. This ensures data integrity even in the event of system failures or concurrent access.
        - COMMIT: To save changes permanently to the database.
        - ROLLBACK: To undo changes made during a transaction if an error occurs or the transaction is not completed successfully.
        - SAVEPOINT: To set a point within a transaction to which you can later roll back.
5. **Database Administration and Optimization:**
    - Indexing: SQL commands are used to create and manage indexes, which significantly improve the performance of data retrieval operations.
    - Views: Creating virtual tables (views) based on the result of a SQL query, simplifying complex queries and providing controlled access to data.
    - Stored Procedures and Functions: Defining reusable blocks of SQL code that can be executed on demand, enhancing efficiency and maintainability.
    - Backup and Restore: While often handled by RDBMS-specific utilities, SQL commands can be part of broader backup and recovery strategies

4. What are the key features of SQL?

Here are some key features of Structured Query Language (SQL):

1. Data Definition Language (DDL): SQL provides a set of commands to define and modify the structure of a database, including creating tables, modifying table structure, and dropping tables.
2. Data Manipulation Language (DML): SQL provides a set of commands to manipulate data within a database, including adding, modifying, and deleting data.
3. Query Language: SQL provides a rich set of commands for querying a database to retrieve data, including the ability to filter, sort, group, and join data from multiple tables.

4. Transaction Control: SQL supports transaction processing, which allows users to group a set of database operations into a single transaction that can be rolled back in case of failure.
5. Data Integrity: SQL includes features to enforce data integrity, such as the ability to specify constraints on the values that can be inserted or updated in a table, and to enforce referential integrity between tables.
6. User Access Control: SQL provides mechanisms to control user access to a database, including the ability to grant and revoke privileges to perform certain operations on the database.
7. Portability: SQL is a standardized language, meaning that SQL code written for one database management system can be used on another system with minimal modification.

LAB EXERCISES:

• Lab 1: Create a new database named school_db and a table called students with the following columns: student_id, student_name, age, class, and address.

Create the database school_db:

```
CREATE DATABASE school_db;


CREATE TABLE STUDENTS
(
    STUDENT_ID INT PRIMARY KEY,
    STUDENT_NAME VARCHAR(50),
    AGE INT,
    CLASS VARCHAR (10),
    ADDRESS VARCHAR (50)
);
```

• Lab 2: Insert five records into the students table and retrieve all records using the SELECT statement.

```
INSERT INTO students (student_id, student_name, age, class, address)
VALUES
```

(1, 'Alice Johnson', 15, '10-A', 'London'),

(2, 'Bob Williams', 16, '11-B', ' London '),

(3, 'Charlie Brown', 15, '10-A', ' London '),

(4, 'Diana Miller', 17, '12-C', ' London '),

(5, 'Edward Davis', 16, '11-B', ' London ');

2. SQL Syntax

Theory Questions:

1. What are the basic components of SQL syntax?

The basic components of SQL syntax can be categorized into five main groups:

1. Data Definition Language (DDL): Used for creating, modifying, and deleting database objects like tables, indexes, and views. Key commands include CREATE, ALTER, and DROP.
2. Data Manipulation Language (DML): Used for interacting with the data within the tables. Key commands are INSERT, UPDATE, and DELETE.
3. Data Query Language (DQL): Used for retrieving data from a database. The primary command is SELECT.
4. Data Control Language (DCL): Used to control access to the data and the database itself. Key commands include GRANT and REVOKE to manage user permissions.
5. Transaction Control Language (TCL): Used to manage transactions to ensure data integrity. Key commands are COMMIT (to save changes), ROLLBACK (to undo changes), and SAVEPOINT (to set a checkpoint within a transaction).

2. Write the general structure of an SQL SELECT statement.

SELECT column1, column2, ...

FROM table_name

[WHERE condition]

[GROUP BY column]

[HAVING condition]

[ORDER BY column [ASC|DESC]];

3. Explain the role of clauses in SQL statements.

in SQL, clauses are fundamental components of a statement that **define the scope, conditions, and specific operations** to be performed on the data. They act like modifiers or qualifiers that refine the main command (like SELECT, INSERT, UPDATE, DELETE) to achieve a precise outcome.

Clauses are the building blocks that allow you to construct complex and precise SQL queries. They provide the necessary context and conditions for the main SQL commands to operate, enabling you to:

- Specify where data comes from.
- Filter unwanted data.
- Group and summarize data.
- Sort the output.
- Control the amount of data returned.
- Define the exact information to be displayed.

Without clauses, SQL would be extremely limited, only capable of performing very basic operations on entire tables. They are what give SQL its power and flexibility in data retrieval and manipulation.

3. SQL Constraints

LAB EXERCISES:

Lab 1: Write SQL queries to retrieve specific columns (student_name and age) from the students table.

SELECT student_name, age FROM students;

Lab 2: Write SQL queries to retrieve all students whose age is greater than 10.

SELECT * FROM students WHERE age > 10;

Theory Questions:

1. What are constraints in SQL? List and explain the different types of constraints.

Constraints in SQL are rules that are applied to columns in a table to limit the type of data that can be inserted, updated, or deleted. They are used to enforce data integrity, ensuring the accuracy and reliability of the data in a database.

Here are the main types of constraints:

- NOT NULL: Ensures that a column cannot contain a NULL value. It forces a value to be present for every row in that column.
- UNIQUE: Ensures that all values in a column are different. While a table can have multiple unique constraints, it can only have one primary key.
- PRIMARY KEY: A combination of NOT NULL and UNIQUE. It uniquely identifies each row in a table. A table can have only one primary key, which cannot contain NULL values.
- FOREIGN KEY: Establishes a link between two tables. It ensures that a value in a column of one table must match a value in the primary key of another table. This maintains referential integrity.
- CHECK: Enforces a condition that all values in a column must satisfy. For example, it can ensure that an age column always contains a value greater than 18.
- DEFAULT: Provides a default value for a column when no value is explicitly specified during an INSERT operation.

2. How do PRIMARY KEY and FOREIGN KEY constraints different?

Feature	PRIMARY KEY	FOREIGN KEY
Purpose	Uniquely identifies records within a single table.	Establishes relationship between tables and enforces referential integrity
Uniqueness	Must be unique.	Can contain duplicates.
Nullability	Cannot be NULL.	Can be NULL
Count per Table	Only one.	Multiple.
Indexing	Automatically indexed	Not automatically indexed.
Role	Defines identify within a table.	Defines relationships between tables.

3. What is the role of NOT NULL and UNIQUE constraints?

NOT NULL Constraint :-

Ensures a column always has a value. No, specifically prohibits NULL values. Does not enforce uniqueness. It enforces that specific columns are mandatory. If a column is defined as NOT NULL, you cannot insert a new row or update an existing row without providing a value for that column.

Example:

CREATE TABLE Products (

```
    ProductID INT NOT NULL,
    ProductName VARCHAR(255) NOT NULL,
    Description VARCHAR(MAX),
    Price DECIMAL(10, 2) NOT NULL
);
```

UNIQUE Constraint :-

It guarantees that no two rows will have the same value for the specified column(s). This is vital for fields that are meant to be unique identifiers, even if they aren't the primary key. A table can have multiple UNIQUE constraints, as long as each constraint applies to a different column or set of columns. Ensures all values in a column are distinct. A PRIMARY KEY implicitly has UNIQUE.

Example :-

```
CREATE TABLE Users (
    UserID INT PRIMARY KEY,
    Username VARCHAR(50) UNIQUE,
    Email VARCHAR(255) UNIQUE,
    RegistrationDate DATE
);
```

LAB EXERCISES:

Lab 1: Create a table teachers with the following columns: teacher_id (Primary Key), teacher_name (NOT NULL), subject (NOT NULL), and email (UNIQUE).

```
CREATE TABLE teachers (
    teacher_id INT PRIMARY KEY,
    teacher_name VARCHAR(100) NOT NULL,
    subject VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE
);
```

CREATE	Creates new database objects (e.g., tables, views, indexes).
ALTER	Modifies the structure of existing objects (e.g., add/remove columns).
DROP	Deletes objects from the database (e.g., tables, databases).
TRUNCATE	Removes all rows from a table, but keeps its structure intact.
RENAME	Renames a database object (e.g., a table).

4. Main SQL Commands and Sub-commands(DDL)

Theory Questions:

1. Define the SQL Data Definition Language (DDL).

DDL (Data Definition Language) is a subset of SQL used to define, modify, and manage the structure of database objects such as tables, schemas, indexes, and constraints.

- DDL commands affect the structure of the database, not the data itself.
- Changes made with DDL are usually auto-committed (saved immediately).
- Used mainly during database design and schema management.

Common DDL Commands:

Examples:

CREATE a table:

```
CREATE TABLE employees (
   emp_id INT PRIMARY KEY,
   name VARCHAR(100),
   position VARCHAR(50)
);
```

ALTER a table:

```
ALTER TABLE employees ADD salary DECIMAL(10,2);
```

DROP a table:

```
DROP TABLE employees;
```

TRUNCATE a table:

```
TRUNCATE TABLE employees;
```

2. Explain the CREATE command and its syntax.

The CREATE command is a core part of SQL's Data Definition Language (DDL). It is used to create new database objects, such as: Tables, Databases, Views, Indexes.

```
CREATE TABLE table_name (
    column1 datatype [constraint],
    column2 datatype [constraint],
    …
);
```

3. What is the purpose of specifying data types and constraints during table creation?

When creating a table in SQL, data types and constraints are essential to ensure data accuracy, integrity, and efficiency in your database.

1. Data Types – Define What Kind of Data Can Be Stored

Purpose: To enforce the type of data that can be stored in each column, which helps in saving space, improving performance, and avoiding invalid data entries.

2. Constraints – Enforce Rules on the Data

Purpose: To maintain data integrity and enforce business rules at the database level.

LAB EXERCISES:

Lab 1: Create a table courses with columns: course_id, course_name, and course_credits. Set the course_id as the primary key.

```
CREATE TABLE courses (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(255) NOT NULL,
    course_credits DECIMAL(4, 2)
);
```

Lab 2: Use the CREATE command to create a database university_db.

```
CREATE DATABASE university_db;
```

5. ALTER Command

Theory Questions:

1. What is the use of the ALTER command in SQL?

The ALTER command in SQL is used to **modify the structure of an existing database object**, typically a table. It allows you to change the table schema without having to delete and recreate the table.

Uses of ALTER in SQL :-

Add a Column: ALTER TABLE employees ADD date_of_birth DATE;

Drop a Column: ALTER TABLE employees DROP COLUMN date_of_birth;

Rename a Column: ALTER TABLE employees RENAME COLUMN name TO full_name;

Modify a Column's Data Type: ALTER TABLE employees MODIFY COLUMN salary DECIMAL(10, 2);

Add Constraints: ALTER TABLE employees ADD CONSTRAINT pk_employee_id PRIMARY KEY (employee_id);

Drop Constraints: ALTER TABLE employees DROP CONSTRAINT pk_employee_id;

Rename a Table: ALTER TABLE employees RENAME TO staff;

2. How can you add, modify, and drop columns from a table using ALTER?

Add a Column: ALTER TABLE employees ADD date_of_birth DATE;

Modify a Column's Data Type: ALTER TABLE employees MODIFY COLUMN salary DECIMAL(10, 2);

Drop a Column: ALTER TABLE employees DROP COLUMN date_of_birth;

LAB EXERCISES:

Lab 1: Modify the courses table by adding a column course_duration using the ALTER command.

ALTER TABLE courses

ADD course_duration INT;

Lab 2: Drop the course_credits column from the courses table.

ALTER TABLE courses

DROP COLUMN course_credits;

6. DROP Command

Theory Questions:

1. What is the function of the DROP command in SQL?

The DROP command in SQL is used to permanently delete database objects such as tables, databases, views, indexes, or stored procedures. Once an object is dropped, all of its data and structure are lost and cannot be recovered.

Uses of the DROP Command :-

1. Drop a Table: This command removes the entire employees table from the database.

→ DROP TABLE employees;

2. Drop a Database: Deletes the entire database, including all its tables and data.

→ DROP DATABASE company_db;

3. Drop a View: Removes a view definition from the database.

→ DROP VIEW employee_view;

4. Drop an Index: Removes an index.

→ DROP INDEX idx_emp_name;

5. Drop a Constraint: Removes a constraint (like a foreign key or primary key) from a table.

→ ALTER TABLE employees DROP CONSTRAINT fk_department;

2. What are the implications of dropping a table from a database?

Dropping a table from a database is a destructive and irreversible operation. Here key implications of dropping:

1. Data Loss : Permanent deletion of all data in the table. Cannot be recovered unless you have a backup. Affects users and applications that rely on this data.

2. Loss of Relationships and Constraints : Foreign key constraints involving the table are dropped. Any relationships with other tables (e.g., via JOINs or keys) are broken. May lead to orphaned records or data integrity issues in related tables.

3. Impact on Dependent Objects : Views, stored procedures, functions, and triggers that reference the table may fail or break. Applications with hard-coded table references may throw errors.

4. Application Failures : Queries, forms, APIs, or UI components expecting the table will likely malfunction or crash. If in production, this can cause service disruption.

5. Loss of Metadata : Table schema, indexes, constraints, and triggers defined on the table are lost. Cannot retrieve the structure or usage history without prior documentation.

6. Security and Audit Implications : Any permissions or access controls tied to the table are lost. In regulated environments, dropping a table without proper audit can lead to compliance issues.

LAB EXERCISES:

Lab 1: Drop the teachers table from the school_db database.

DROP TABLE teachers;

Lab 2: Drop the students table from the school_db database and verify that the table has been removed.

Drop the table:

DROP TABLE students;

Verify the table has been removed:

SELECT * FROM students;

7. Data Manipulation Language (DML)

Theory Questions:

1. Define the INSERT, UPDATE, and DELETE commands in SQL.

The INSERT, UPDATE, and DELETE commands in SQL, which are part of the Data Manipulation Language (DML) used to manage data in relational databases:

INSERT : Used to add new rows (records) into a table.

Syntax:

INSERT INTO table_name (column1, column2, ...)

VALUES (value1, value2, ...);

Example:

INSERT INTO employees (id, name, position)

VALUES (1, 'Alice Johnson', 'Manager');

UPDATE : Used to modify existing records in a table.

Syntax:

UPDATE table_name

SET column1 = value1, column2 = value2, ...

WHERE condition;

Example:

UPDATE employees

SET position = 'Senior Manager'

WHERE id = 1;

DELETE: Used to remove existing records from a table.

Syntax:

DELETE FROM table_name

WHERE condition;

Example:

DELETE FROM employees

WHERE id = 1;

2. What is the importance of the WHERE clause in UPDATE and DELETE operations?

The WHERE clause in UPDATE and DELETE operations is critically important because it defines which rows in a table should be affected.

→ Importance in UPDATE Operations :

- Targeted Modifications: It allows you to update only the rows that meet a specified condition.

- Preventing Accidental Data Corruption: Without WHERE, you risk overwriting vast amounts of data indiscriminately, leading to incorrect information across your entire dataset.
- Data Integrity and Accuracy: By precisely controlling which rows are updated, you maintain the accuracy and consistency of your database.

→ Importance in DELETE Operations :

- Selective Row Removal: It enables you to delete only the rows that satisfy a specific condition.
- Avoiding Mass Data Loss: Similar to UPDATE, the WHERE clause in a DELETE statement will result in the deletion of all records in the table. This is often irreversible without a database backup.
- Maintaining Referential Integrity (Foreign Keys): When deleting rows, the WHERE clause ensures you're removing only specific records, which is crucial when dealing with foreign key relationships. Deleting all parent records without considering child records could lead to data if ON DELETE CASCADE is not set up.

LAB EXERCISES:

Lab 1: Insert three records into the courses table using the INSERT command.

```
INSERT INTO courses (course_id, course_name, course_credits)
VALUES (101, 'Introduction to Databases', 3.0);
INSERT INTO courses (course_id, course_name, course_credits)
VALUES (102, 'Data Structures and Algorithms', 4.0);
INSERT INTO courses (course_id, course_name, course_credits)
VALUES (103, 'Web Development Fundamentals', 3.5);
```

Lab 2: Update the course duration of a specific course using the UPDATE command.

```
UPDATE courses SET course_duration = 45 WHERE course_id = 101;
```

Lab 3: Delete a course with a specific course_id from the courses table using the DELETE command.

```
DELETE FROM courses WHERE course_id = 101;
```

8. Data Query Language (DQL)

Theory Questions:

1. What is the SELECT statement, and how is it used to query data?

The SELECT statement is the most fundamental and frequently used command in SQL. Its primary purpose is to retrieve data from one or more tables in a relational database. It allows you to specify exactly what data you want to see, from which tables, and under what conditions.

Here some clauses are use with SELECT Statement :

WHERE : Filters rows based on condition

ORDER BY : Short results (ASC or DESC)

GROUP BY : Groups rows for aggregation

JOIN : Combines rows from multiple tables

DISTINCT : Removes duplicate rows

2. Explain the use of the ORDER BY and WHERE clauses in SQL queries.

The ORDER BY and WHERE clauses in SQL are essential tools for **filtering** and **organizing** data returned by a query.

ORDER BY Clause — Sorting Rows : The ORDER BY clause sorts the result set based on one or more columns. You can sort in ascending (ASC) or descending (DESC) order.

Syntax:

SELECT column1, column2 FROM table_name ORDER BY column1 [ASC|DESC];

Example:

SELECT * FROM employees ORDER BY hire_date DESC;

Multiple Columns in ORDER BY:

SELECT * FROM employees ORDER BY department ASC, salary DESC;

WHERE Clause — Filtering Rows: The WHERE clause filters records before any sorting or grouping happens. It specifies which rows should be returned based on a condition.

Syntax:

SELECT column1, column2 FROM table_name WHERE condition;

Example:

SELECT * FROM employees WHERE department = 'Marketing';

LAB EXERCISES:

Lab 1: Retrieve all courses from the courses table using the SELECT statement.

SELECT * FROM courses;

Lab 2: Sort the courses based on course_duration in descending order using ORDER BY.

SELECT * FROM courses ORDER BY course_duration DESC;

Lab 3: Limit the results of the SELECT query to show only the top two courses using LIMIT.

SELECT * FROM courses LIMIT 2;

9. Data Control Language (DCL)

Theory Questions:

1. What is the purpose of GRANT and REVOKE in SQL?

GRANT — Give Permissions

Purpose: To give specific privileges to a user or role, such as the ability to SELECT, INSERT, UPDATE, or DELETE data, or manage other permissions.

REVOKE — Remove Permissions

Purpose: To take away previously granted privileges from a user or role.

2. How do you manage privileges using these commands?

Managing privileges in SQL using GRANT and REVOKE involves carefully assigning and removing access to database objects.

Using GRANT to Assign Privileges:

Syntax:

GRANT privilege[, ...] ON object TO user_or_role;

Using REVOKE to Remove Privileges :

Syntax:

REVOKE privilege[, ...] ON object FROM user_or_role;

LAB EXERCISES:

Lab 1: Create two new users user1 and user2 and grant user1 permission to SELECT

from the courses table.

CREATE USER 'user1'@'localhost' IDENTIFIED BY 'password123';

CREATE USER 'user2'@'localhost' IDENTIFIED BY 'password456';

Lab 2: Revoke the INSERT permission from user1 and give it to user2.

Revoke the INSERT permission from user1:

REVOKE INSERT ON courses FROM 'user1'@'localhost';

Grant the INSERT permission to user2:

GRANT INSERT ON courses TO 'user2'@'localhost';

10. Transaction Control Language (TCL)

Theory Questions:

1. What is the purpose of the COMMIT and ROLLBACK commands in SQL?

In SQL, COMMIT and ROLLBACK are transaction control commands used to manage changes made by SQL statements. They are essential for maintaining data integrity in a database.

1. COMMIT : The purpose of the Commit is to save all the changes made by the current transaction to the database permanently. Once a COMMIT is issued, the changes cannot be undone. Used when all operations in a transaction have completed successfully and you want to make the changes permanent.

2. ROLLBACK : The purpose of the Rollback is to undo all changes made by the current transaction. Used when an error occurs or a condition is not met, and you want to restore the database to its previous state.

2. Explain how transactions are managed in SQL databases.

Transactions in SQL databases are managed using a set of principles and commands to ensure data consistency, integrity, and isolation, even in multi-user environments. A transaction is a sequence of one or more SQL operations (like INSERT, UPDATE, DELETE) executed as a single logical unit of work.

LAB EXERCISES:

Lab 1: Insert a few rows into the courses table and use COMMIT to save the changes.

INSERT INTO courses (course_id, course_name, course_duration)

VALUES

(104, 'Database Administration', 50),

(105, 'Cloud Computing Basics', 40),

(106, 'Cybersecurity Fundamentals', 60);

COMMIT;

Lab 2: Insert additional rows, then use ROLLBACK to undo the last insert operation.

INSERT INTO courses (course_id, course_name, course_duration)

VALUES (107, 'Advanced SQL Queries', 55);

Use ROLLBACK to undo the insert:

ROLLBACK;

Verify the change was undone:

SELECT * FROM courses WHERE course_id = 107;

Lab 3: Create a SAVEPOINT before updating the courses table, and use it to roll back specific changes.

Start a transaction and make an initial change:

START TRANSACTION;

INSERT INTO courses (course_id, course_name, course_duration)

VALUES (201, 'Data Science Essentials', 70);

Create a SAVEPOINT:

SAVEPOINT before_update;

Perform the update operation:

UPDATE courses

SET course_name = 'Data Science and Machine Learning'

WHERE course_id = 201;

Rollback to the SAVEPOINT:

ROLLBACK TO before_update;

Commit the transaction:

COMMIT;

11. SQL Joins

Theory Questions:

1. Explain the concept of JOIN in SQL. What is the difference between INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL OUTER JOIN?

In SQL, a JOIN is used to combine rows from two or more tables based on a related column between them (typically a foreign key relationship). It allows you to query data across multiple tables as if they were one.

INNER JOIN → Only Alice and Bob (customers with orders)

LEFT JOIN → Alice, Bob, Charlie (Charlie has no order, so NULLs for product)

RIGHT JOIN → Alice, Bob, and the order with customer_id 4 (no matching customer)

FULL OUTER JOIN → All customers and all orders, even unmatched ones (Charlie and customer_id 4)

2. How are joins used to combine data from multiple tables?

Joins are used in SQL to combine data from multiple tables by matching rows based on related columns, typically primary key and foreign key relationships. This allows you to create complex queries that pull together relevant data spread across normalized tables.

How Joins Work : Each row in one table is paired with rows in another table where a specified condition (usually equality) is met. This is usually done using the JOIN clause along with an ON condition.

LAB EXERCISES:

Lab 1: Create two tables: departments and employees. Perform an INNER JOIN to display employees along with their respective departments.

Create the departments table:

```
CREATE TABLE departments (
    dept_id INT PRIMARY KEY,
```

```
    dept_name VARCHAR(50)
);
```

Create the employees table:

```
CREATE TABLE employees (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(50),
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);
```

Insert data into both tables:

Insert into departments

```
INSERT INTO departments (dept_id, dept_name)
VALUES
(1, 'Sales'),
(2, 'IT'),
(3, 'HR');
```

Insert into employees

```
INSERT INTO employees (emp_id, emp_name, dept_id)
VALUES
(101, 'John Doe', 1),
(102, 'Jane Smith', 2),
(103, 'Peter Jones', 1),
(104, 'Mary Brown', 3),
(105, 'David Lee', 2);
```

Perform the INNER JOIN:

```
SELECT e.emp_name, d.dept_name FROM employees e INNER JOIN departments d ON
e.dept_id = d.dept_id;
```

Lab 2: Use a LEFT JOIN to show all departments, even those without employees.

```
SELECT
   d.dept_name,
   e.emp_name
FROM
   departments d
LEFT JOIN
   employees e ON d.dept_id = e.dept_id;
```

12. SQL Group By

Theory Questions:

1. What is the GROUP BY clause in SQL? How is it used with aggregate functions?

The GROUP BY clause in SQL is used to group rows that have the same values in specified columns into summary rows, like totals, counts, or averages. It's most commonly used with aggregate functions to perform calculations on each group.

The true power of GROUP BY comes when it's combined with aggregate functions. Aggregate functions perform a calculation on a set of rows and return a single value. When used with GROUP BY, they operate on *each group* of rows independently.

Common aggregate functions include:

- COUNT(): Returns the number of rows in a group.
- SUM(): Calculates the sum of values in a numeric column for each group.
- AVG(): Calculates the average of values in a numeric column for each group.
- MIN(): Finds the minimum value in a column for each group.
- MAX(): Finds the maximum value in a column for each group.

2. Explain the difference between GROUP BY and ORDER BY.

Feature	GROUP BY	ORDER BY
Primary Goal	Summarize/Aggregate data	Sort/Arrange data

Feature	GROUP BY	ORDER BY
Rows	Reduces the number of rows (to groups)	Does not change the number of rows
Columns	Used with aggregate functions on non-grouped columns	Can use any column in the SELECT or original table
Position	Typically comes after WHERE, before HAVING & ORDER BY	Always the last clause in a SELECT statement (logically)
Impact	Changes the granularity of the result	Only affects the presentation of the result

LAB EXERCISES:

Lab 1: Group employees by department and count the number of employees in each department using GROUP BY.

```
SELECT d.dept_name, COUNT(e.emp_id) AS number_of_employees FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id GROUP BY d.dept_name;
```

Lab 2: Use the AVG aggregate function to find the average salary of employees in each department.

```
SELECT d.dept_name,  AVG(e.salary) AS average_salary FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id GROUP BY d.dept_name;
```

13. SQL Stored Procedure

Theory Questions:

1. What is a stored procedure in SQL, and how does it differ from a standard SQL query?

A stored procedure in SQL is a precompiled set of SQL statements (and optional control-flow logic) that is stored in the database and can be executed repeatedly. It functions like a subroutine or function in programming languages and is commonly used to encapsulate business logic, improve performance, and promote reusability.

Feature	**Stored Procedure**	**Standard SQL Query**
Definition	A reusable, named block of SQL code	A single SQL statement or simple script

Feature	Stored Procedure	Standard SQL Query
Reusability	Reusable and can be called multiple times	Usually written and run once
Parameters	Can accept input/output parameters	Usually does not use parameters directly
Control Flow	Supports complex logic (loops, conditionals)	Does not support procedural logic
Performance	Can be precompiled for faster execution	Parsed and compiled at runtime
Security	Can encapsulate logic and limit user access	Exposes logic and structure directly

2. Explain the advantages of using stored procedures.

Advantages of using stored procedure :

1. Improved Performance

- Precompiled Execution: Stored procedures are compiled once and stored in the database. Subsequent executions are faster because they reuse the compiled plan.
- Reduced Network Traffic: Executing a procedure involves a single call, even if it contains multiple SQL statements. This reduces the amount of data transmitted between the application and the database server.

2. Reusability and Maintainability

- Code Reuse: You can define business logic once and reuse it in multiple applications or modules.
- Centralized Logic: Keeping logic in the database makes maintenance easier since updates happen in one place rather than across multiple client applications.

3. Security and Access Control

- Granular Permissions: Users can be granted permission to execute a stored procedure without needing direct access to the underlying tables.
- Data Abstraction: Stored procedures can expose specific functionality while hiding the internal details and structure of the database.

4. Reduced Risk of SQL Injection

- Parameterized Queries: Stored procedures typically use parameters, which help prevent SQL injection attacks by separating SQL logic from data inputs.

5. Consistency and Integrity

- Enforced Business Rules: Procedures can enforce business logic consistently across applications, reducing the risk of errors or inconsistencies.
- Transactional Control: They can include logic for beginning, committing, or rolling back transactions, ensuring data integrity.

6. Support for Complex Logic

- Control Structures: Stored procedures support variables, loops, conditional logic (IF, CASE), and error handling, making them suitable for complex operations.

LAB EXERCISES:

Lab 1: Write a stored procedure to retrieve all employees from the employees table based on department.

```
CREATE PROCEDURE GetEmployeesByDepartment
   @department_id INT
AS
BEGIN
   SELECT e.emp_id, e.emp_name, d.dept_name FROM employees e
   INNER JOIN
      departments d ON e.dept_id = d.dept_id
   WHERE
      e.dept_id = @department_id;
END;
```

Lab 2: Write a stored procedure that accepts course_id as input and returns the course details.

```
CREATE PROCEDURE GetCourseDetails
   @course_id INT
AS
BEGIN
   SELECT
      course_id,
```

```
    course_name,
    course_duration
  FROM
    courses
  WHERE
    course_id = @course_id;
END;
```

14. SQL View

Theory Questions:

1. What is a view in SQL, and how is it different from a table?

A view in SQL is a virtual table based on the result of a SELECT query. It doesn't store data physically like a table does — instead, it presents data dynamically from one or more tables.

Feature	View	Table
Storage	Virtual – no physical storage of data	Physically stores data
Definition	Defined by a SELECT query	Defined by column names and data types
Data Updates	Usually not updatable (depends on DB & query)	Fully updatable
Performance	Slower for large views; recalculates on access	Faster for reads/writes as data is stored
Use Case	Used for simplifying complex queries, securing data	Used for storing actual application data
Security	Can restrict access to specific columns or rows	Needs permissions on the full table
Dependency	Depends on underlying table(s)	Independent storage unit

2. Explain the advantages of using views in SQL databases.

Here are the main advantages of using views in SQL databases:

1. Simplifies Complex Queries : Views can hide complex JOIN, GROUP BY, and filtering logic behind a simple name. Users can query a view like a regular table without needing to understand the underlying SQL.

2. Improves Security : You can restrict users to access only specific columns or rows of a table through a view. Prevents direct access to sensitive data while allowing limited access as needed.

3. Provides Logical Data Independence : If the database schema changes (e.g., column renamed, tables split), views can provide a consistent interface to users or applications. Applications that use the view don't need to be updated if the underlying tables change, as long as the view is maintained.

4. Encourages Reusability and Maintainability : Once created, a view can be reused in multiple queries or reports. Changes to the view's definition update all queries using that view.

5. Supports Data Abstraction : Views can present a customized version of the data for different users or applications. Useful in creating business-specific representations of data without altering the base tables.

6. Reduces Data Redundancy : Views do not store data physically — they are just saved SQL queries. Saves storage space compared to creating duplicate tables or summary tables.

7. Can Improve Performance (in Some Cases) : In some databases, materialized views (views that store data physically) can improve performance for complex and frequently used queries.

LAB EXERCISES:

Lab 1: Create a view to show all employees along with their department names.

```
CREATE VIEW employees_with_departments AS
SELECT
    e.emp_id,
    e.emp_name,
    d.dept_name
FROM
    employees e
INNER JOIN
```

```
    departments d ON e.dept_id = d.dept_id;
```

Lab 2: Modify the view to exclude employees whose salaries are below $50,000.

```
CREATE OR REPLACE VIEW employees_with_departments AS
SELECT
    e.emp_id,
    e.emp_name,
    e.salary, -- Assuming a salary column exists in the employees table
    d.dept_name
FROM
    employees e
INNER JOIN
    departments d ON e.dept_id = d.dept_id
WHERE
    e.salary >= 50000;
```

15. SQL Triggers

Theory Questions:

1. What is a trigger in SQL? Describe its types and when they are used.

A trigger is a special type of stored procedure that automatically executes in response to certain events on a table or view, such as:

- INSERT
- UPDATE
- DELETE

Types of trigger and their use :-

Type	Description	When It's Used
BEFORE Trigger	Executes before the triggering operation (INSERT, UPDATE, DELETE)	To validate or modify data before it’s written to the table
AFTER Trigger	Executes after the triggering operation is completed	For logging, audit trails, or cascading changes

Type	Description	When It's Used
INSTEAD OF Trigger	Replaces the standard action (used mainly on views)	To allow updates on views or to customize behavior
ROW-level Trigger	Executes once for each row affected	When you want to take action per affected row
STATEMENT-level Trigger	Executes once per SQL statement, regardless of number of rows affected	For general tasks like logging or maintaining summary tables

2. Explain the difference between INSERT, UPDATE, and DELETE triggers.

Trigger Type	Fired When	Accesses	Common Use Cases
INSERT	On new rows	NEW values	Logging new data, validation, defaults
UPDATE	On data change	NEW and OLD	Compare changes, enforce rules, audit
DELETE	On row deletion	OLD values	Track deletions, prevent accidental delete

LAB EXERCISES:

Lab 1: Create a trigger to automatically log changes to the employees table when a new employee is added.

```
CREATE TRIGGER trg_log_employee_insert
ON employees
AFTER INSERT
AS
BEGIN
   INSERT INTO employee_log (employee_id, action_type)
   SELECT
      inserted.emp_id,
      'New Employee Added'
   FROM
      inserted;
END;
```

Lab 2: Create a trigger to update the last_modified timestamp whenever an employee record is updated.

Add the last_modified column to the employees table:

```
ALTER TABLE employees
ADD last_modified DATETIME;
```

Create the Trigger

```
CREATE TRIGGER trg_update_employee_timestamp
BEFORE UPDATE ON employees
FOR EACH ROW
BEGIN
   SET NEW.last_modified = NOW();
END;
```

16. Introduction to PL/SQL

Theory Questions:

1. What is PL/SQL, and how does it extend SQL's capabilities?

PL/SQL (Procedural Language/Structured Query Language) is Oracle's procedural extension to SQL. It combines SQL's data manipulation power with the control structures of traditional programming languages like IF-ELSE, LOOP, and EXCEPTION handling.

It extends SQL's capabilities by:

- Adding Procedural Constructs: It introduces control structures like IF-THEN-ELSE and loops (FOR, WHILE) that are absent in standard SQL, enabling more complex logic.
- Encapsulation and Reusability: Developers can create stored procedures, functions, and triggers, which are reusable units of code stored in the database. This promotes modularity and reduces code duplication.
- Error Handling: It provides a robust exception-handling mechanism (EXCEPTION...WHEN...) to gracefully manage errors during execution.
- Variables and Data Structures: It allows the use of variables, constants, and complex data structures like records and collections, enabling the manipulation of data within the program.

- Performance Improvement: By executing procedural logic directly on the database server, PL/SQL minimizes network traffic between the application and the database, leading to better performance for data-intensive operations.

2. List and explain the benefits of using PL/SQL.

The key benefits of using PL/SQL are:

- Improved Performance: PL/SQL reduces network traffic by allowing you to send a single block of code to the database server, which can contain multiple SQL statements. This is much more efficient than sending each SQL statement individually.
- Enhanced Functionality: It adds procedural programming features (loops, conditional statements, variables) to SQL, enabling developers to create more complex and powerful business logic that goes beyond simple data retrieval and manipulation.
- Code Reusability and Modularity: You can create stored procedures, functions, and packages, which are reusable code units stored in the database. This promotes modular design and reduces code duplication across applications.
- Robust Error Handling: PL/SQL includes a built-in exception-handling mechanism, allowing you to write code that can gracefully manage and respond to errors, preventing application crashes.
- Security: By using stored procedures, you can restrict user access to the underlying tables and only grant permissions to execute the procedures, adding a layer of security.
- Portability: PL/SQL is fully portable across different Oracle environments, meaning code written on one system can be easily deployed on another.

LAB EXERCISES:

Lab 1: Write a PL/SQL block to print the total number of employees from the employees table.

```
SET SERVEROUTPUT ON;
DECLARE
   v_total_employees NUMBER;
BEGIN
```

```
    SELECT COUNT(*)
    INTO v_total_employees
    FROM employees;
END;
/
```

Lab 2: Create a PL/SQL block that calculates the total sales from an orders table.

```
SET SERVEROUTPUT ON;
DECLARE
    v_total_sales NUMBER;
BEGIN
    SELECT SUM(order_amount)
    INTO v_total_sales
    FROM orders;
END;
/
```

17. PL/SQL Control Structures

Theory Questions:

1. What are control structures in PL/SQL? Explain the IF-THEN and LOOP control structures.

Control structures in PL/SQL are used to control the flow of execution in a program.
They help you add logic, conditions, and repetition (loops) to your SQL-based code, similar to traditional programming languages.

IF-THEN: This is the simplest conditional statement. It executes a sequence of statements only if a boolean condition is met.

```
IF condition THEN
    statements;
END IF;
```

LOOP...END LOOP: This is the basic loop structure. It repeatedly executes a sequence of statements until an EXIT statement is encountered.

```
LOOP
   statements;
   EXIT WHEN condition; -- Exits the loop when the condition is true
END LOOP;
```

WHILE-LOOP: This loop executes a block of statements as long as a condition remains true. The condition is checked at the beginning of each iteration.

```
WHILE condition THEN
   statements;
END LOOP;
```

FOR-LOOP: This loop iterates over a specified range of integers. It is a more compact way to perform a set number of iterations.

```
FOR counter_variable IN 1..10 LOOP
   statements;
END LOOP;
```

2. How do control structures in PL/SQL help in writing complex queries?

Control structures in PL/SQL enable you to write complex queries by providing the procedural logic that standard SQL lacks.

how they help:

- Conditional Logic (IF-THEN-ELSE): You can construct queries that change their behavior based on certain conditions. For example, you might select data from different tables, apply different filters, or calculate values differently based on a variable or a condition evaluated within your PL/SQL block.
- Iterative Processing (Loops): Loops allow you to process data one row at a time or perform an operation a specific number of times. This is especially useful for tasks that involve iterating over the results of a query (using a cursor) to perform a specific action on each row, like complex data updates or generating reports.
- Variables and Dynamic SQL: Control structures work hand-in-hand with variables to create dynamic queries. You can use loops to build a complex query string piece by

piece, adding different WHERE clauses or JOIN conditions based on logic, and then execute that dynamically built query.

LAB EXERCISES:

Lab 1: Write a PL/SQL block using an IF-THEN condition to check the department of an employee.

```
SET SERVEROUTPUT ON;
DECLARE
   v_employee_name   employees.emp_name%TYPE;
   v_department_name departments.dept_name%TYPE;
   v_employee_id     employees.emp_id%TYPE := 102;
BEGIN
   -- Fetch the employee's name and department name
   SELECT
      e.emp_name,
      d.dept_name
   INTO
      v_employee_name,
      v_department_name
   FROM
      employees e
   INNER JOIN
      departments d ON e.dept_id = d.dept_id
   WHERE
      e.emp_id = v_employee_id;
   IF v_department_name = 'IT' THEN
      DBMS_OUTPUT.PUT_LINE(v_employee_name || ' works in the IT department.');
   ELSE
```

```
      DBMS_OUTPUT.PUT_LINE(v_employee_name || ' works in the ' ||
v_department_name || ' department, not IT.');
   END IF;

EXCEPTION
   WHEN NO_DATA_FOUND THEN
      DBMS_OUTPUT.PUT_LINE('Employee with ID ' || v_employee_id || ' not found.');
END;
/
```

Lab 2: Use a FOR LOOP to iterate through employee records and display their names.

```
SET SERVEROUTPUT ON;
DECLARE

BEGIN

   FOR emp_rec IN (
      SELECT emp_name
      FROM employees
   ) LOOP
      -- For each row, print the employee's name.
      DBMS_OUTPUT.PUT_LINE('Employee Name: ' || emp_rec.emp_name);
   END LOOP;
END;
/
```

18. SQL Cursors

Theory Questions:

1. What is a cursor in PL/SQL? Explain the difference between implicit and explicit cursors.

In PL/SQL, a **cursor** is a pointer to a private SQL work area in memory that stores information about a SELECT statement and the rows it returns.

The two types of cursors are:

- **Implicit Cursors:** These are automatically created and managed by the Oracle database for every INSERT, UPDATE, DELETE, or SELECT INTO statement that is not explicitly handled by a programmer-defined cursor. They are ideal for queries that are expected to return a single row or for DML statements. You have no direct control over their lifecycle (opening, fetching, and closing), but you can access their status through attributes like SQL%ROWCOUNT (to see how many rows were affected) or SQL%FOUND (to check if a row was found).
- **Explicit Cursors:** These are manually declared and managed by the developer to handle SELECT statements that are expected to return multiple rows. Explicit cursors give you full control over the process. You must explicitly:
    1. DECLARE the cursor with a name and an associated SELECT statement.
    2. OPEN the cursor to execute the query and populate the result set.
    3. FETCH rows from the cursor one by one into local variables or records.
    4. CLOSE the cursor to release the memory and resources it was using.

2. When would you use an explicit cursor over an implicit one?

You would use an explicit cursor over an implicit one in the following situations:

- When a query is expected to return multiple rows: Explicit cursors are designed to handle and process a result set one row at a time.[1] This is essential for iterating through a list of records and applying custom logic to each one.[2]
- For fine-grained control: Explicit cursors provide you with full control over the cursor's lifecycle—you decide exactly when to OPEN, FETCH, and CLOSE it.[3] This is crucial for managing database resources efficiently and performing operations like fetching a limited number of rows or stopping processing early based on a condition.[4]
- When you need to perform row-level operations: If you need to perform complex logic, such as conditional updates or logging, on each individual row returned by a query, an explicit cursor allows you to fetch each row into a variable and manipulate it before moving to the next.
- To avoid exceptions for no data found: While an implicit SELECT...INTO statement raises a NO_DATA_FOUND exception if no rows are returned, an explicit cursor allows

you to check for this condition using the %NOTFOUND attribute without raising an exception, which can be more efficient in certain scenarios.

LAB EXERCISES:

Lab 1: Write a PL/SQL block using an explicit cursor to retrieve and display employee details.

```
SET SERVEROUTPUT ON;
DECLARE
  CURSOR c_employees IS
    SELECT emp_id, emp_name
    FROM employees;
  v_emp_id   employees.emp_id%TYPE;
  v_emp_name employees.emp_name%TYPE;
BEGIN
  OPEN c_employees;
  LOOP
    FETCH c_employees INTO v_emp_id, v_emp_name;
    EXIT WHEN c_employees%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE('Employee ID: ' || v_emp_id || ', Name: ' || v_emp_name);
  END LOOP;
  CLOSE c_employees;
END;
/
```

Lab 2: Create a cursor to retrieve all courses and display them one by one.

```
SET SERVEROUTPUT ON;
DECLARE
  CURSOR c_courses IS
    SELECT course_id, course_name
    FROM courses;
```

```
    v_course_id   courses.course_id%TYPE;
    v_course_name courses.course_name%TYPE;
BEGIN
    OPEN c_courses;
    LOOP
        FETCH c_courses INTO v_course_id, v_course_name;
        EXIT WHEN c_courses%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE('Course ID: ' || v_course_id || ', Course Name: ' ||
v_course_name);
    END LOOP;
    CLOSE c_courses;
END;
/
```

19. Rollback and Commit Savepoint

Theory Questions:

1. Explain the concept of SAVEPOINT in transaction management. How do ROLLBACK and COMMIT interact with savepoints?

In transaction management, a SAVEPOINT is a named marker or a checkpoint within an active transaction.[1] It allows you to divide a long transaction into smaller, manageable parts. The main purpose of a savepoint is to provide the ability to perform a partial rollback.[2]

How it interacts with ROLLBACK and COMMIT:

- ROLLBACK:
    - A standard ROLLBACK statement (without a savepoint) undoes all changes made in the current transaction, returning the database to its state before the transaction began.[3] It discards all savepoints.
    - ROLLBACK TO SAVEPOINT savepoint_name; allows you to undo only the changes made *after* that specific savepoint was created.[4] All work performed before the savepoint is preserved, and the transaction remains active, allowing you to continue with new operations or set new savepoints.[5] Any savepoints created *after* the target savepoint are also discarded.[6]
- COMMIT:

- A COMMIT statement permanently saves all changes made in the entire transaction.[7]
- When you COMMIT, it finalizes the entire transaction, making all changes (including those before and after any savepoints) permanent.[8] All savepoints within that transaction are automatically discarded. You cannot roll back to a savepoint after a COMMIT has been executed.

2. When is it useful to use savepoints in a database transaction?

Savepoints are useful in a database transaction when you need to perform a partial rollback.[1] Instead of undoing the entire transaction, a savepoint allows you to revert only a portion of the changes made since that specific point.[2]

Use of savepoints in a database transaction :

- Handling Errors: If an error occurs in the middle of a complex, multi-statement transaction, you can roll back to a savepoint to undo the faulty operation without canceling all the work done before it.[3]
- Complex Business Logic: In long transactions involving multiple steps, savepoints can serve as logical checkpoints.[4] For example, you could set a savepoint after a successful data insertion and then proceed with a subsequent update.[5] If the update fails, you can roll back to the savepoint, keeping the successful insertion.
- Conditional Processing: When a transaction's next steps depend on a previous outcome, you can use a savepoint to try a specific action. If it fails, you can roll back to the savepoint and try an alternative.

LAB EXERCISES:

Lab 1: Perform a transaction where you create a savepoint, insert records, then rollback to the savepoint.

START TRANSACTION;

Insert Initial Records

INSERT INTO products (product_id, product_name, price) VALUES (1, 'Laptop', 1200.00);

INSERT INTO products (product_id, product_name, price) VALUES (2, 'Mouse', 25.00);

Create a Savepoint

SAVEPOINT before_updates;

Insert More Records (These will be rolled back)

```
INSERT INTO products (product_id, product_name, price) VALUES (3, 'Keyboard', 75.00);
```

```
INSERT INTO products (product_id, product_name, price) VALUES (4, 'Monitor', 300.00);
```

Rollback to the Savepoint

```
ROLLBACK TO before_updates;
```

Lab 2: Commit part of a transaction after using a savepoint and then rollback the remaining changes.

```
BEGIN;
```

```
INSERT INTO employees (id, name) VALUES (1, 'Alice');
```

```
SAVEPOINT sp1;
```

```
INSERT INTO employees (id, name) VALUES (2, 'Bob');
```

```
INSERT INTO employees (id, name) VALUES (3, 'Charlie');
```

```
ROLLBACK TO sp1;
```

```
COMMIT;
```